

When a Model Request Is Not the Model Run

The substitution that shaped this toolkit’s verification, and the patch that answers it

A subagent asks for `claude-fable-5`. The request is correct at every layer you can see from your own machine: the raw argument on the launch, the model the harness resolved after applying its precedence rules, the model name the interface shows you when you open the subagent. And the subagent is served `claude-opus-5` for some or all of its turns. There is no error. There is no warning. Nothing in the transcript announces it. The only record that reveals what happened is the model field the API itself writes onto each response.

This is the incident the toolkit’s whole verification architecture was built around. It is why a model claim here is settled by an audit of a transcript and never by a configuration file, an interface header, or the model’s own account of itself. It is also why the toolkit ships instruments rather than advice: the failure is invisible to attention and cheap to catch mechanically, which is the exact shape that calls for machinery.

Two boundaries keep the problem in proportion. The substitution affects only subagent requests that target `claude-fable-5`. Every other model id was served faithfully in every era measured, 31 out of 31 across days, and 100 percent of non-fable ids in the first population examined. The main conversation loop was never substituted at all.

The second boundary decides everything that follows. Prevention lives on the serving side and is out of user reach; nothing you can set on your own machine stops this. What is in reach is verification, and verification turned out to be cheap, because the substitution, when it happens at all, happens at a fixed and early moment. A subagent’s fate is observable by roughly its fifth call. Every countermeasure below follows from that one property.

Which record answers “what ran?”

Four records describe a subagent’s model, and they are not four opinions about the same thing. Each one records something real, and only one of them records what answered.

The first is the **raw argument**: the model name passed on the launch, stored in the subagent’s own metadata file and absent when the launch omitted the name. It is an input record.

The second is the **resolved model**: the harness’s launch-time resolution after it applies the four-rank precedence, written onto the launch row of the main transcript. It is the only field carrying the `[1m]` display suffix, and it is evidently the source of what the interface header shows you. This is intent rather than serving, and the difference is not academic: it diverged from the serving record in about 94 percent of fable-resolved launches during the measured incident.

The third is the **serving stamp**: the model field the API response itself carries, written per call by the serving side onto every assistant turn of the subagent’s transcript. This is the one authoritative layer, because it is the only record downstream of what actually answered.

The fourth is **asking the subagent what it is running on**, and it is disqualified as an instrument. It was wrong three times out of five when measured. A subagent assembles that answer from its own loaded context, so it repeats whatever its documents call the default, which is precisely the belief under test.

One witness sits outside the transcript record entirely: the account’s usage-by-model metering. It cannot attribute tokens to a particular run, but a model carrying tokens you never requested is a serving-side fact that no client-side layer can manufacture.

From this comes the rule the toolkit enforces everywhere. **A model claim is verified by the serving stamp or it is not verified.** Headers, configurations, resolutions and the model’s own testimony are statements of intent or belief. They help you diagnose why a substitution happened; none of them is evidence of what ran. The moment to remember it is the moment you are about to attribute some behavior

— quality, error rate, speed, cost — to a named model without having read that run’s stamps. Read them first.

The check is one line against a subagent’s transcript:

```
grep -oh '"model":"claude-[~]*"' <child-transcript.jsonl> | sort | uniq -c
```

The `claude-` prefix is load-bearing. On a transcript that launches subagents, each launch’s model *argument* sits in the same field shape, so a bare short name appearing in that field is never a serving stamp. The API stamps full ids only.

What the record shows

The substitution arrives in two shapes. Either the substituted model answers from call one, which was the character of the earliest era and of full-tool subagents generally, or the run swaps mid-course after exactly three faithful turns and stays substituted. That second shape held 7 out of 7 runs across two days and in every swap of one controlled evening campaign. You see it by reading the per-turn stamps in order. The consequence matters more than the curiosity: the swap point is fixed and early, which is the only reason early certification is possible.

Two subagents launched in the same breath can diverge. Three identical-shape subagents of one pinned agent, launched in a single message within the same second, came back different: one faithful for 12 turns, one swapped at call 4 after three faithful turns and substituted for 8 more, one faithful for 19. Everything a client can control is shared inside that trio — the configuration files, the environment, the harness version, the agent definition, the launch instant. The decision is therefore made per request, on the serving side. No local state can explain a split inside one message.

The rate drifts at the scale of hours, on frozen configuration. The measured series runs: 0 out of 32 across 2 and 3 August 2026, every subagent substituted from its first call; 45 out of 69, or 65.2 percent, inside the window from 14:53 to about 23:14 on 4 August, with a sub-window at 84.2 percent, 16 of 19; 1 out of 9 on 5 August; a move from about 50 percent to about 90 percent across 100 minutes on 6 August with nothing changed; and then 10 out of 10 faithful across roughly 13 hours from late on 6 August to the morning of the 7th. Each figure is a tally of serving stamps re-derived from the full transcript record. The consequence rides every number quoted anywhere: a rate is a property of a configuration *and an hour*, never of a configuration alone.

One request-side discriminator was isolated by single-variable experiment. Nine scratch agents were written with identical minimal bodies, each carrying the same pinned model and launched with no model argument so that the pin governed; all nine resolved as intended. Against a base grant of reading and searching, four variants each came back faithful two runs out of two: the base itself, the base plus the shell, plus the web tools, and plus the ability to launch children — which exonerated the prime suspect. Adding the Skill tool substituted three times out of three, on every stamp. A separate bisection padded the agent body at a faithful five-tool grant across initial requests of roughly 69, 75, 81 and 87 thousand tokens and stayed faithful four times out of four, which exonerates initial request size up to about 87 thousand for that shape; and a run whose context grew to about 146 thousand tokens mid-course stayed faithful for all 11 of its turns, so growth after the start is free.

That result is graded carefully. The Skill grant is measured **sufficient** to flip a correctly resolved request, at $n=3$. It is not claimed **necessary**: one counterexample was substituted at about 94 thousand tokens of initial request with no Skill grant and with three grants each measured faithful, which leaves a possible size-like condition somewhere above 90 thousand and the band between 87 and 94 thousand unprobed.

And the discriminator is itself conditioned on the regime. In the faithful window spanning 6 and 7 August, a two-run control on the very `Read`, `Grep`, `Skill` shape that had substituted in every prior era served faithfully. In that window the discriminator did not discriminate. Treat it as the best request-side bet available and not as a rule the serving side is obliged to honor.

A swap boundary is not always terminal. One subagent served faithfully for three calls, then substituted for 167, then returned to the requested model for its final two. Read as one continuous stamp sequence, it says that a verdict taken from the first row alone, or from the last row alone, can be wrong in either direction. Audit every turn.

A separate false-positive class blocks the obvious workaround. Headless sessions whose main model is `claude-fable-5` were refused at turn one under a biology-hazard category on a benign kickoff — instructing one subagent to read eight local project files and report their line counts. The refusal repeated three times, across fresh sessions, reworded text, and a front-door file six times smaller, with no biological content anywhere in the prompt or the project context. The same kickoff under a different main model runs clean. Since the main loop is the one path never substituted, routing fable work through headless main sessions would have been the natural way around the whole problem, and this class closes it.

That refusal is shaped by the prompt surface, and it clears. Running `claude -p "Execute the instructions in the file <path>"` went through cleanly, with the main loop served as requested for 36 turns and its executor subagent for 13, both faithful, where the identical instructions written inline in the prompt had been refused three times out of three. The classifier fires on the inline wording rather than on the meaning, so moving the same text behind a file reference restores the route.

The vendor’s own metering corroborates the substitution. In the primary investigation session, which requested `claude-fable-5` for the main loop and for virtually every subagent, the session’s usage-by-model table showed `claude-opus-5` carrying more output tokens than fable itself, 4.1 million against 2.9 million out. That face is independent of every transcript. It means a substitution window shows up as the substituted model’s token line carrying work you never routed to it, and the crossover when a regime turns faithful is the signature.

The client-side configuration space was swept, and came back empty. One evening, one regime window, two runs per arm: forcing the model through the top-rank environment variable, including in its `[1m]` form; leaving that variable unset; naming a short model name on the launch; pinning in the agent file as a full id, as a short name, and in `[1m]` form; inheriting from the session; remapping the default model alias; and running effort at maximum against high. No arm moved fidelity beyond regime noise. Two sub-findings are worth carrying: the `[1m]` request form cannot be placed on a subagent request at all, since it normalizes to the plain id at the environment rank, at the agent-file rank, and through the remap — only the main-loop picker carries the form, and the main loop is the one path never substituted; and the resolution rank is exonerated, since ranks two, three and four all served faithfully on restricted-grant agents in the same window.

Every local location that could affect resolution was read directly and excluded. Managed settings do not exist on the machine. The shell startup files carry no relevant variables. The project and local settings never carried a model key in any commit or dated snapshot. No allowlist, override or fallback keys are present. No hook rewrites the launch input, which a direct search over both hook directories confirmed with zero hits. The mechanism is server-side. The settings axis is closed, and there is no local defect to fix.

The patch: verification, not prevention

Six pieces compose into a working shape. None of them prevents a substitution, because prevention is not available to a user; together they make a substituted run either impossible to complete unnoticed or cheap to discard.

A pinned, restricted-shape executor agent. Its frontmatter carries the full model id, which sits at the third rank of precedence and was measured to govern — nine probe cells out of nine resolved from their pins with no launch argument. It is granted reading, editing, writing, two kinds of searching, and the shell. It is granted neither the Skill tool nor the ability to launch children. It is launched with no model argument, so that the pin decides. One precondition holds the whole arrangement up: the top-rank environment variable must stay at `inherit` or unset, because set to a model id it overrides both the launch argument and the pin.

The reasoning behind that tool grant is worth separating, because its three parts rest on three different grades of evidence and the toolkit keeps them apart on purpose. The Skill tool is excluded on a **measured trigger** — it is the flipper isolated above. The ability to launch children is excluded as a **design choice** rather than on measurement, since it measured faithful twice out of two: an executor spawns nothing. And the grant as a whole rests on an **untested composition** — each tool in it was measured faithful individually, and the five-tool set was measured faithful as a set four times out of four, but the six-tool composition was never its own experimental cell, and neither was the size band above 90 thousand tokens. The shape is a measured bet, not a guarantee, and it is stated that way everywhere it appears.

An executor without the Skill tool still reaches skill content, by read-pointer: the brief names the skill file and the subagent reads it itself, which also satisfies the standing rule that a worker reads its own sources. On grade, the executor is **fixture-measured** — a five-launch acceptance passed with every assistant turn of all five runs stamped as requested. The real-world qualifier belongs beside it: the route reduces substitution and does not eliminate it. The running tally recorded substituted runs on this shape, including one substituted from its first row. Audit every stamp.

Pointed reads that double as the warmup. A fable-tier brief opens the task with a few small pointed reads — the persona file, the skill file — so that the transcript crosses the deterministic swap boundary on real working context instead of on throwaway calls. The reads that certify the launch are the same reads that begin the work, so certification costs nothing extra. The delivery mechanics are **measured-working**: eight subagents out of eight named and applied the framework from the file they were pointed at, and all eight were served as requested. In the composition test, six out of six were faithful and six out of six showed verified uptake across persona-only, skill-only and combined arms, with four of the six certified live at call five while still mid-task.

A watchdog that reads the opening stamps. It returns a verdict by roughly call five: FAITHFUL, exit code 0, meaning certified, continue; SWAPPED at call k , exit code 1, meaning relaunch, or proceed knowingly and log that you did; UNDETERMINED, exit code 2, meaning investigate. It runs live against a still-running subagent with `--watch`, or after the fact against a finished transcript, and an `--expect` argument generalizes it to any pinned model rather than to one id. Its grade is **live-measured**: eight production certifications out of eight issued mid-run by call five at trivial cost, on top of seven fixture cases out of seven and correct out-of-sample verdicts on live transcripts.

The economics of relaunching. Because the verdict lands at about the fifth call, a substituted launch is discarded seconds in rather than discovered after a long run has been spent on the wrong model. At the rates measured during the campaign the pattern cost about one to two launches per certified subagent — and that figure is conditioned on the regime, so it moves as the rate moves.

A completion flag that asks nothing of you. A hook fires when a subagent finishes, compares that subagent’s serving stamp against the shipped model promise of its route — the fable executor promises `claude-fable-5`, the supervised executor promises `claude-opus-5` — and raises a served-model substitution finding on any disagreement. The check is gated on the value rather than on any text shape: only a real serving stamp may fire it, because a launch argument or an inference from a pin is intent and not serving. Generic subagents carry no promise and remain audited by the coordinator. On a certified route the subagent either works on the model it was promised or the swap is announced on the spot, and silence stops being one of the outcomes. Its grade is **fixture-measured** — 94 guard cases out of 94, including a red arm proving that the pre-build version stayed silent — with the honest limit that no real swap has crossed it live yet, because the regime has stayed faithful since it shipped.

A dispatch gate that makes the shape unavoidable. When a fable-tier or supervised-tier subagent is about to launch, the gate denies the launch unless the launch names a readable brief file or opens with the warmup token, the resolved brief carries a role line and either names a persona file or states plainly that no specialist fits, and a fable-tier brief carries the warmup token. The denial comes back as a structured reason naming the failed check, the missing element and the fix, because a correct relaunch is the point rather than a blocked one. A companion advisory hook prints the ready-to-run certification command as soon as a launch returns, so the next action needs no recall. The gate is **live-measured**: a briefless launch was denied in a real fresh session with the denial text captured, across 66 guard cases out of 66, with the

pre-build silent pass printed as the red arm.

One caveat travels with every number above. These rates are conditioned on the regime and none of them is a property of a configuration alone. **Certification at launch, not configuration, carries the guarantee** — no combination of settings was found that moves subagent fidelity, while the fidelity itself keeps moving. A correct request is not yet the run you asked for, so audit the stamps.

Where this stands, and what it asks of you

The underlying defect is open and sits on the serving side. It has been reported to the vendor with per-request receipts and a runnable reproduction package; the vendor report and the incident record are held privately. Everything above is a measured construction *around* the defect rather than a fix *for* it, and it can stop working without notice if the serving side changes.

Three habits carry it. **Audit the serving stamps** of every model-sensitive subagent when you collect its work; that is the entire discipline, one search or one run of the shipped audit instrument per subagent that matters. **Check your usage-by-model face** periodically, since a model carrying output tokens you never requested is the external tell and shows up even when no one thought to audit a transcript. And **re-run the five-launch acceptance after any Claude Code upgrade**: this environment's own records show behavior changing within a single version, and the version correlation broken at both ends, with degradation beginning before a version bump and opposite outcomes on the same version less than an hour apart. Version stability cannot be assumed.

The instruments arrive with the toolkit installed into `~/.claude` and the kit's own installer run in your project root. In the project's `.claude/` you will find `agents/fable-executor.md` and `agents/opus5-executor.md`, the two certified routes; `skills/model-verification/`, holding `fable_watchdog.py` for live and after-the-fact certification and `model_run_audit.py` for the whole-session audit of intent against what was served, with an exit code you can gate on, beside its `SKILL.md`; `hooks/fable-dispatch-gate.sh`, the launch gate; and `hooks/fable-launch-scaffold.sh`, the advisory scaffold. The completion flag rides the subagent completion hook the toolkit installs at `~/.claude/hooks/subagent_qa_gate.sh`.

For the general model behind the grades used here — what `fixture-measured`, `measured-working` and `live-measured` each license a claim to say — see `ASSURANCE_ARCHITECTURE.md`. This document supplies one worked instance of that model rather than restating it.

What holds all of it together is a single reversal. The serving stamp is the only record of what ran; the substitution is server-side, conditioned on time, and invisible to every layer you can inspect from your own machine. So the answer is to certify early rather than to configure carefully — and a correct request is not yet the run you asked for.